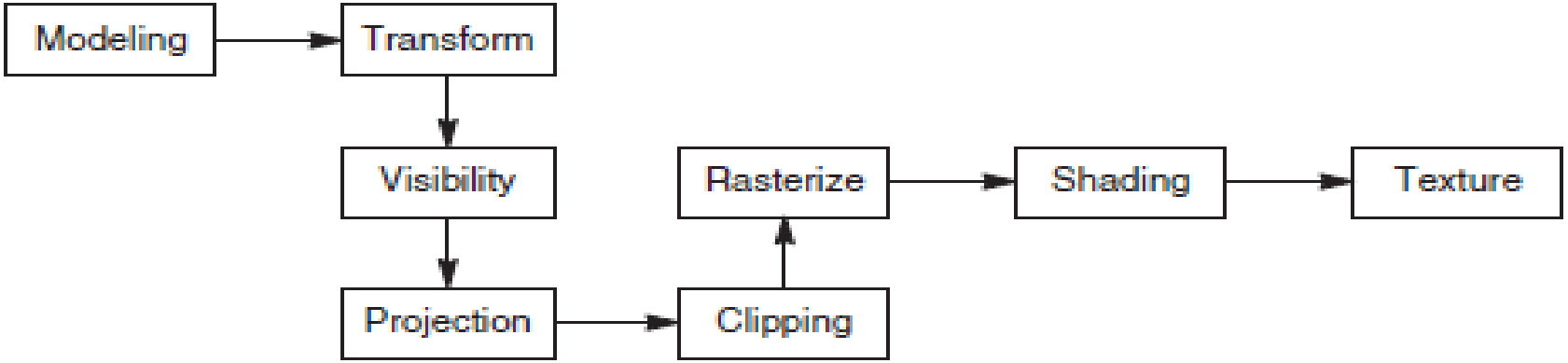
Лабораторная номер 2

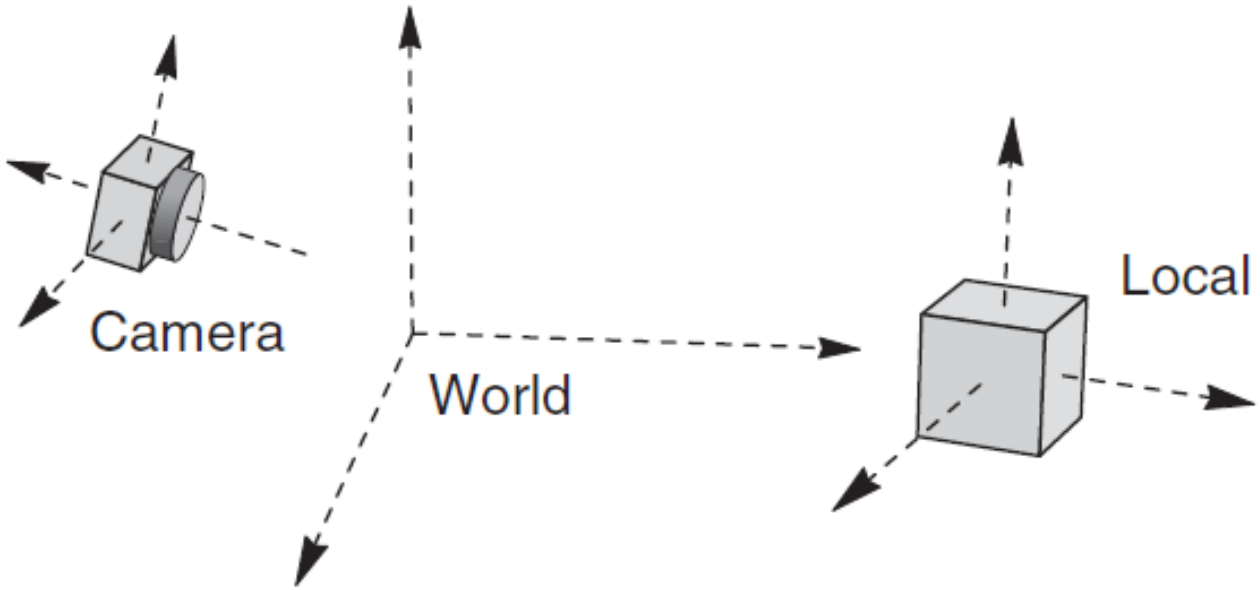
Модельно-видовые и проективные пеобразования

Графический конвейер



- отображение (rendering):
  - преобразования (transformation) – задание местоположения;
  - определение видимости (visibility) – область видимости (field of view) + нелицевые поверхности → отсечение (clipping);
  - проекция на картинную плоскость (projection);
  - растеризация (rasterization);
  - закраска (shading);
  - текстурирование (texturing).

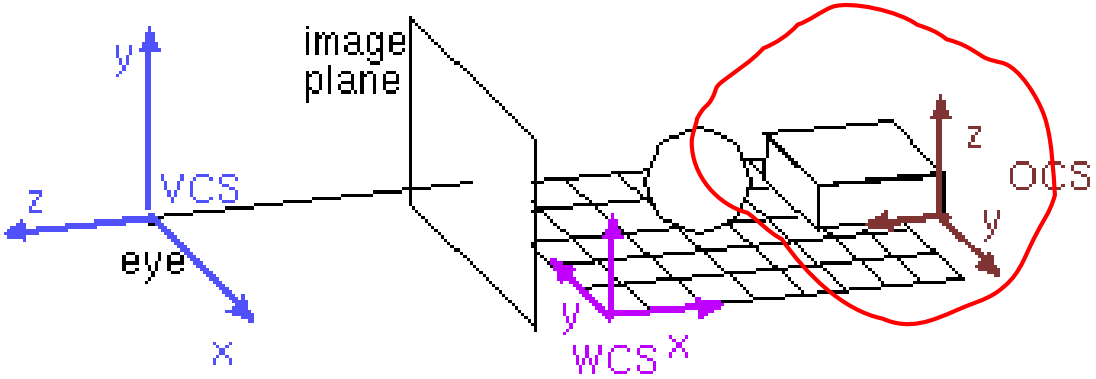
Преобразования координат: этап 1



- модельно-видовые преобразования (преобразование координат моделей отдельных объектов, заданных в локальных системах координат, к некоторой единой (мировой) системе координат и учет положения наблюдателя) → мировые координаты

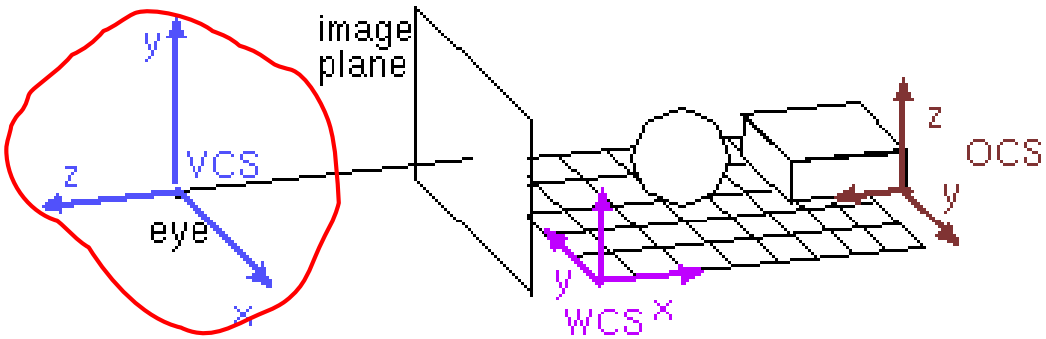
Модельное преобразование

- переводит модель, заданную в локальных (собственных) координатах, в глобальное (мировое пространство);
- сцена «собирается» из частей, с помощью модельных преобразований (обычно композиция переносов, поворотов, масштабирований);
- на выходе – модель в единых *мировых координатах.*



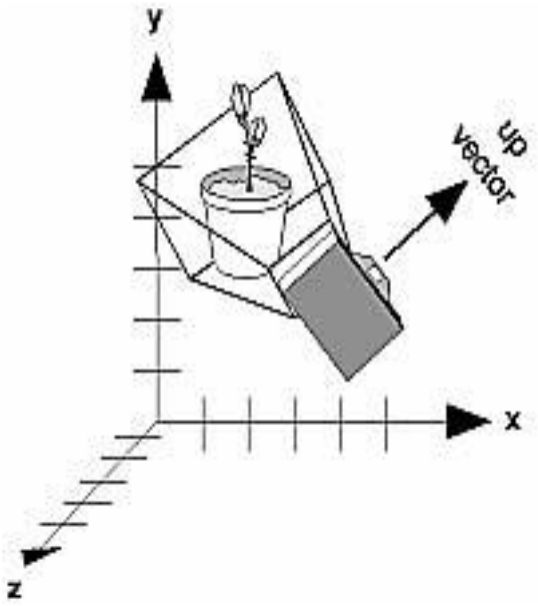
Виртуальная камера

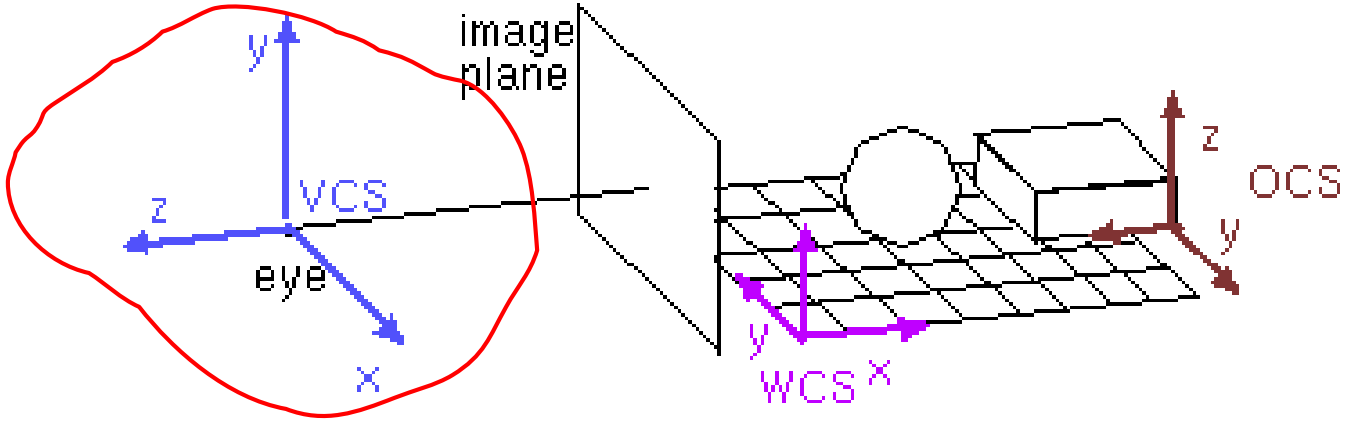
- определяет положение наблюдателя в пространстве
- параметры:
  - положение (точка) наблюдения;
  - направление взгляда;
  - направление «вверх»;
  - параметры проекции;
- положение, направление взгляда и направление «вверх» задают видовое преобразование.



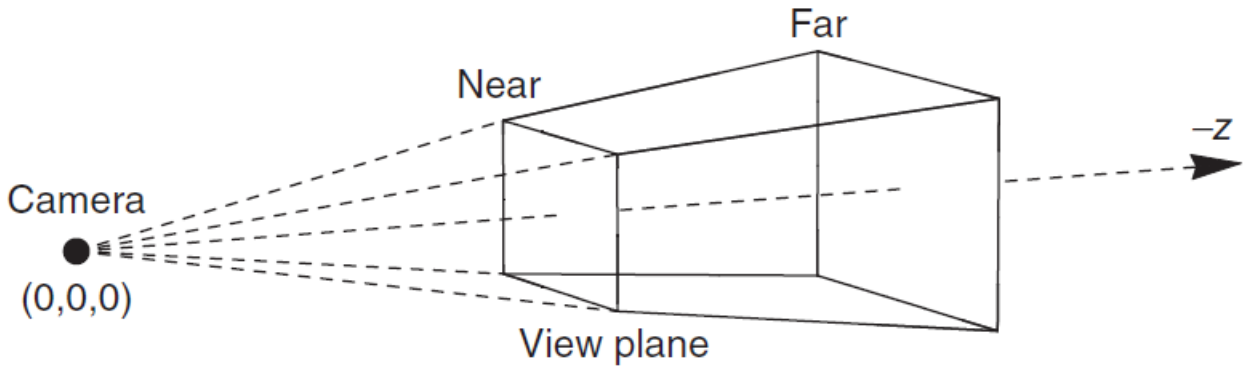
Видовое преобразование

- «подгоняет» мир под стандартную камеру, преобразует мировую систему координат в видовые координаты (которые подходят для «стандартной» камеры);
- на выходе – модель, готовая к проекции на экран.





Преобразования координат: этап 2



- преобразования проецирования (в т.ч. отсечение по объему видимости) → усеченные координаты
- преобразование нормализации (приведение координат к некоторому предопределенному интервалу, например, [-1.0; 1.0]) → нормализованные координаты
- преобразование к порту просмотра (преобразование рабочей станции - из аппаратно-независимой формы в координаты устройства) → оконные координаты

Матричное представление геометрических преобразований

- Координаты представляются вектором-столбцом
- Геометрическое преобразование задается матрицей, умножаемой справа на вектор-столбец координат

  P' = M*P

- Матрица композиции преобразований является произведением матриц элементарных преобразований (матрицы перемножаются в обратном порядке): операция является ассоциативной, но в общем случае некоммутативной

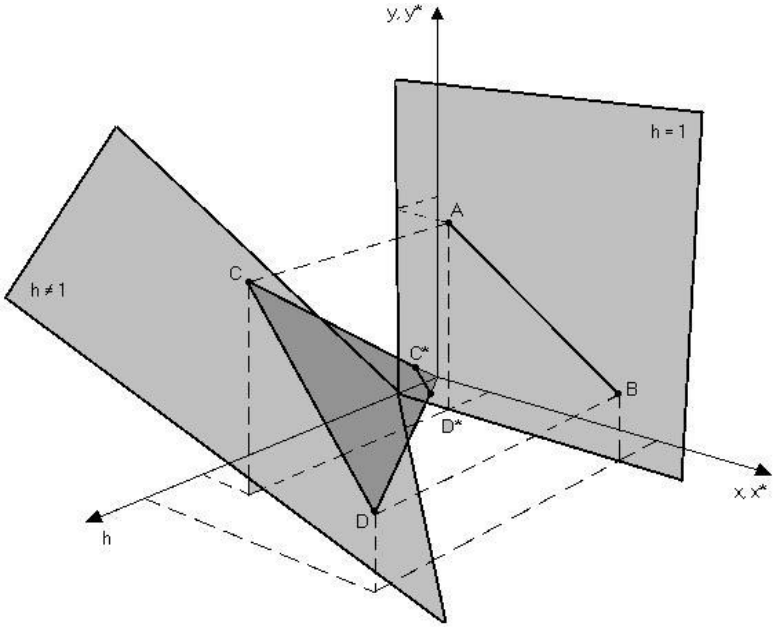
$$M = M_2 * M_1$$

- Обратное преобразование задается обратной матрицей

$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ m & n & p & q \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Типичные линейные преобразования

- общие линейные преобразования
  - w’ != 1 (проективные, необходимо «перспективное деление»)
  - прямые переходят в прямые
- аффинные преобразования
  - w’ = 1
  - сохраняется параллельность линий
  - пример: сдвиг
- преобразование подобия
  - сохраняются углы
  - пример: равномерное масштабирование
- изометрия (движение)
  - сохраняются расстояния
  - пример: поворот, перенос



$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ m & n & p & q \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Аффинные преобразования

Отображение $f : \mathbb{R}^n \to \mathbb{R}^n$ есть **аффинное преобразование**, если найдётся обратимая матрица M и вектор $v \in \mathbb{R}^n$ такие, что

$$f(x) = M \cdot x + v.$$

- прямая переходит в прямую, плоскость – в плоскость;
- сохраняется параллельность прямых и плоскостей;
- конечные (истинные) точки отображаются в конечные;
- сохраняются относительные пропорции;
- сохраняются аффинные комбинации точек.

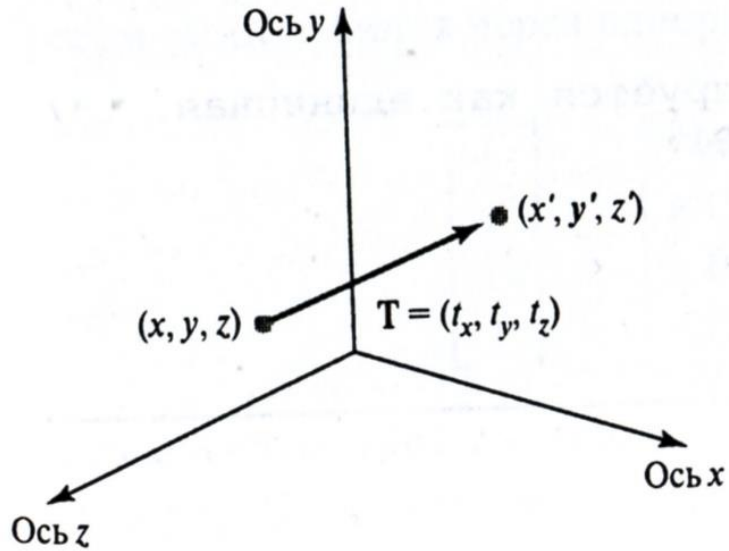
$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Трехмерный перенос

$$x' = x + \Delta x$$

$$y' = y + \Delta y$$

$$z' = z + \Delta z$$



$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & \Delta x \\ 0 & 1 & 0 & \Delta y \\ 0 & 0 & 1 & \Delta z \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

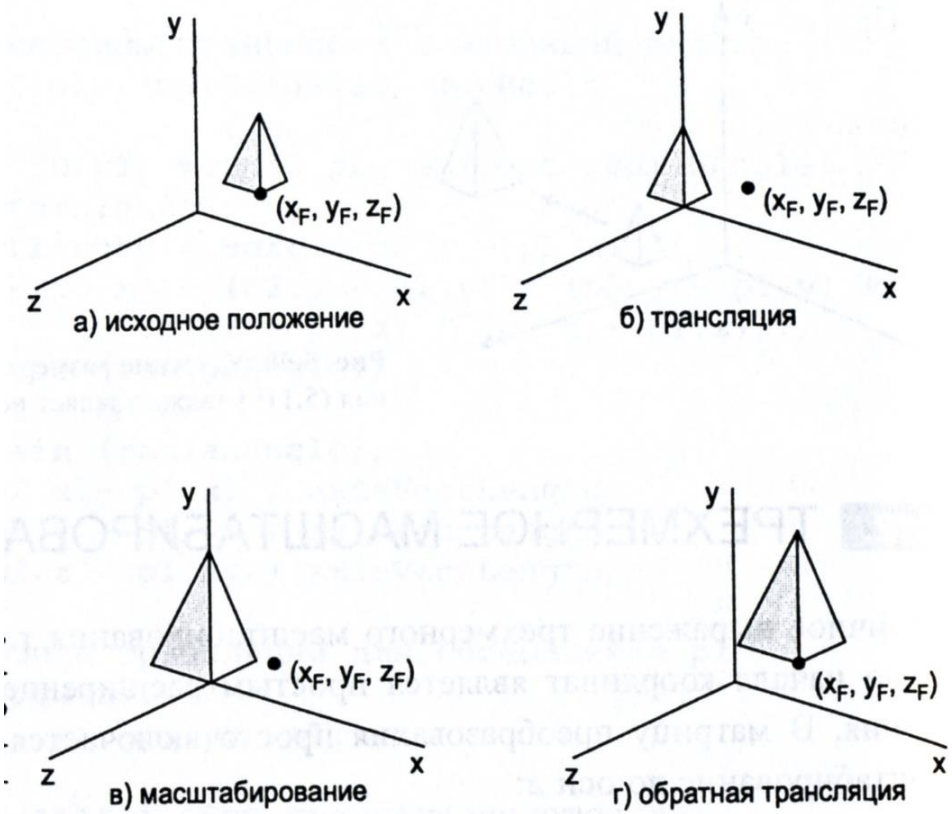
Трехмерное масштабирование

$$x' = ax$$

$$y' = by$$

$$z' = cz$$

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} a & 0 & 0 & 0 \\ 0 & b & 0 & 0 \\ 0 & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$



частные случаи:

- равномерное (пропорциональное) масштабирование;
- отражение от координатной плоскости

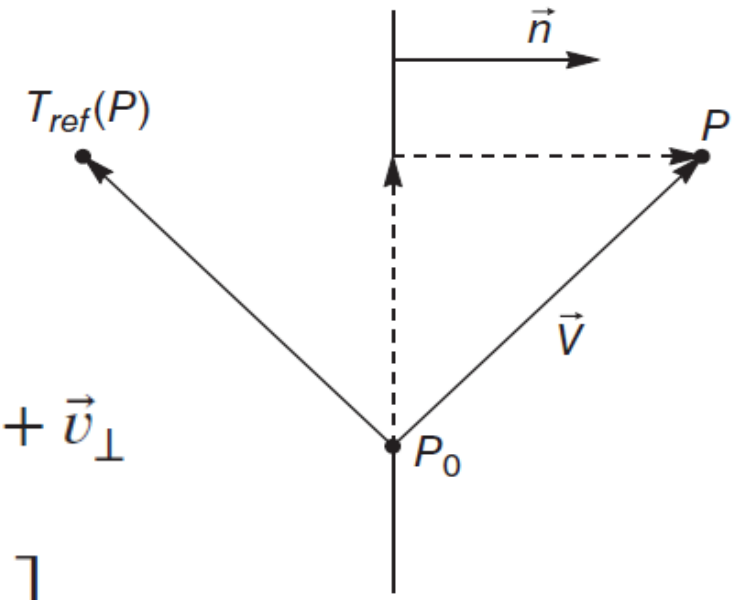
Отражение от произвольной плоскости

$$\vec{v} = \vec{v}_{\parallel} + \vec{v}_{\perp}$$

$$\vec{v}_{\parallel} = (\vec{v} \cdot \vec{n})\vec{n}$$

$$\vec{v}_{\perp} = \vec{v} - (\vec{v} \cdot \vec{n})\vec{n}$$

$$T_{ref}(\vec{v}) = T_{ref}(\vec{v}_{\parallel}) + T_{ref}(\vec{v}_{\perp}) = -\vec{v}_{\parallel} + \vec{v}_{\perp}$$



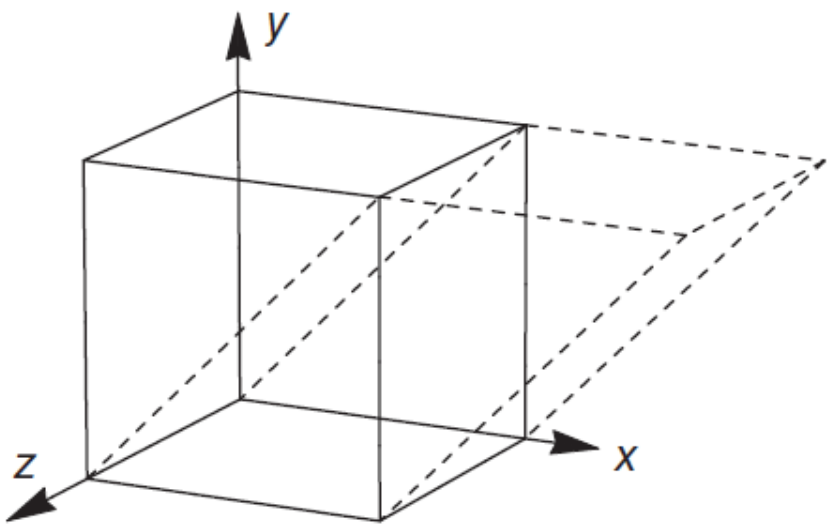
$$M = \vec{n}\vec{n}^T = \begin{bmatrix} n_x^2 & n_x n_y & n_x n_z \\ n_y n_x & n_y^2 & n_y n_z \\ n_z n_x & n_z n_y & n_z^2 \end{bmatrix}$$

$$T_{ref}(\vec{v}) = \vec{v} - 2(\vec{v} \cdot \vec{n})\vec{n} = I\vec{v} - 2M\vec{v} = (I - 2M)\vec{v}$$

Трехмерный сдвиг

$$x' = x + ay$$

$$y' = y + bx$$



$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} 1 & a & 0 & 0 \\ b & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

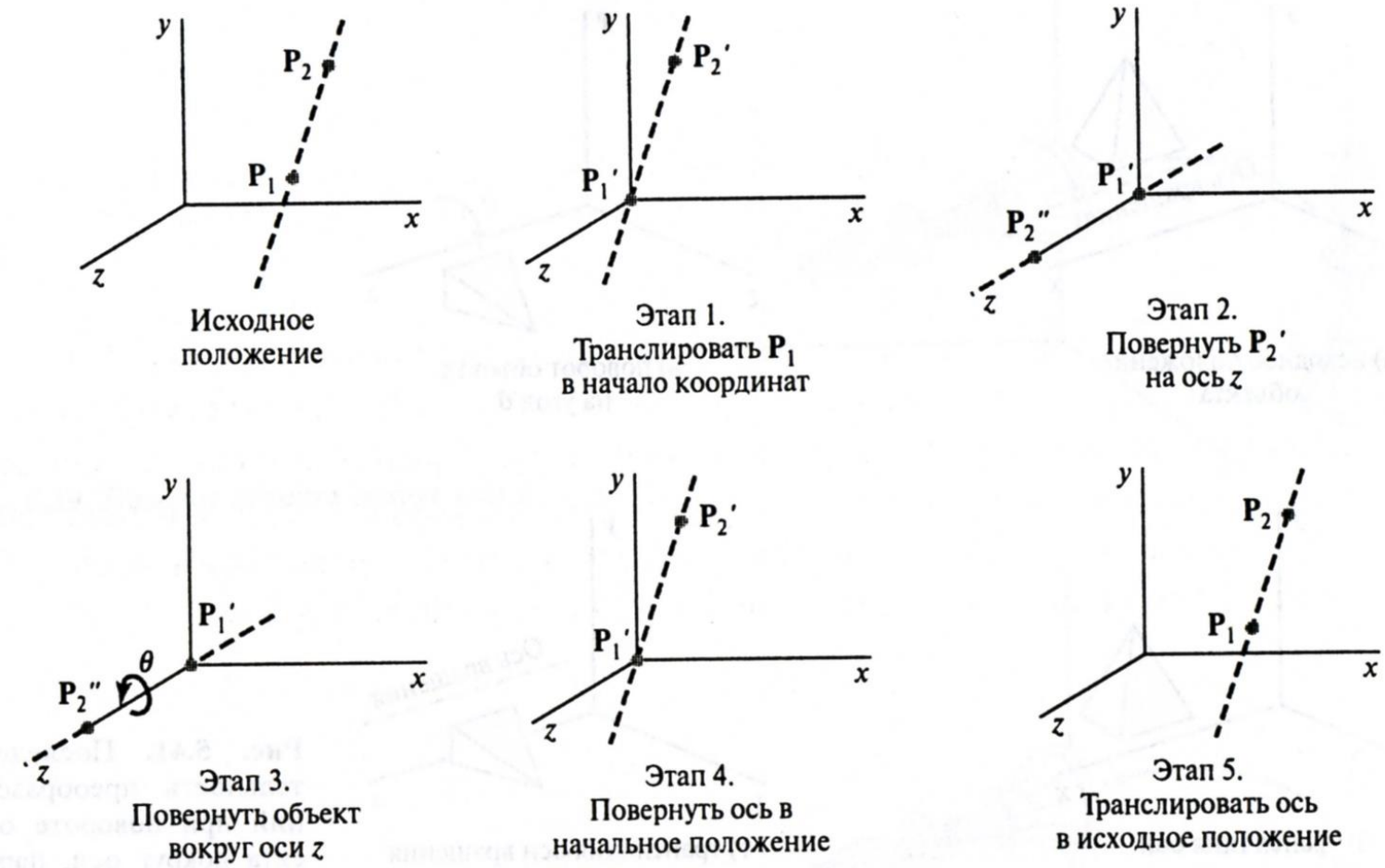
Трехмерный поворот (относительно одной из осей)

$$R_{OZ} = \begin{pmatrix} \cos\varphi & -\sin\varphi & 0 & 0 \\ \sin\varphi & \cos\varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_{OY} = \begin{pmatrix} \cos\varphi & 0 & \sin\varphi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\varphi & 0 & \cos\varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_{OX} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\varphi & -\sin\varphi & 0 \\ 0 & \sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

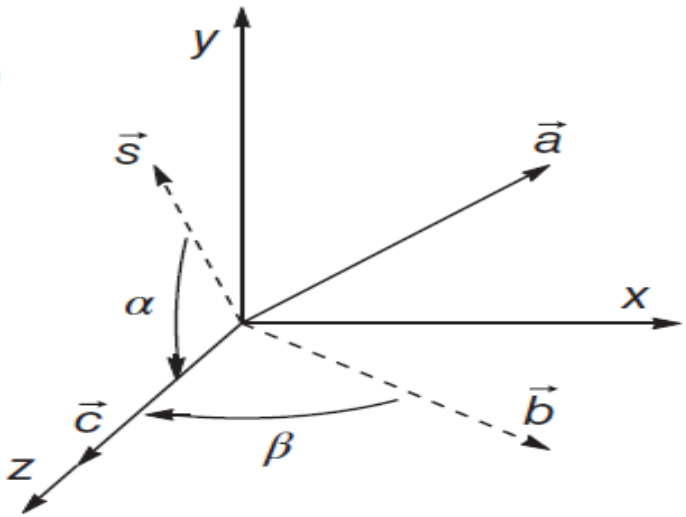
Трехмерный поворот



- Эйлер: комбинация любого числа поворотов всегда эквивалентна единственному повороту вокруг фиксированной оси

Трёхмерный поворот (относительно произвольной оси)

$$M_{arb} = M_x^{-1}(\alpha)M_y^{-1}(\beta)M_z(\theta)M_y(\beta)M_x(\alpha)$$



$$c = \cos\theta$$

$$s = \sin\theta$$

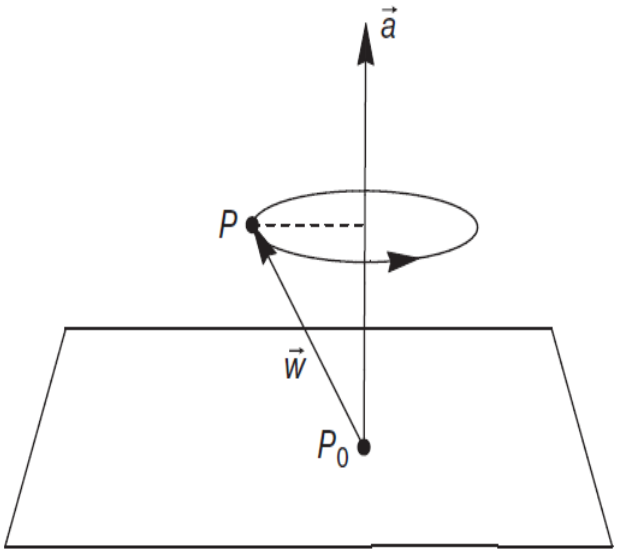
$$M_{arb} = \begin{bmatrix} c + (1-c)a_x^2 & (1-c)a_x a_y - s a_z & (1-c)a_x a_z + s a_y \\ (1-c)a_x a_y + s a_z & c + (1-c)a_y^2 & (1-c)a_y a_z - s a_x \\ (1-c)a_x a_z - s a_y & (1-c)a_y a_z + s a_x & c + (1-c)a_z^2 \end{bmatrix}$$

Трёхмерный поворот (относительно произвольной оси) (2)

$$\vec{w}_{\parallel} = (\vec{a} \cdot \vec{w})\vec{a}$$

$$\vec{w}_{\perp} = \vec{w} - (\vec{a} \cdot \vec{w})\vec{a}$$

$$\vec{w}_r = (\vec{w} - (\vec{a} \cdot \vec{w})\vec{a})\cos\theta + (\vec{a} \times \vec{w}_{\perp})\sin\theta$$

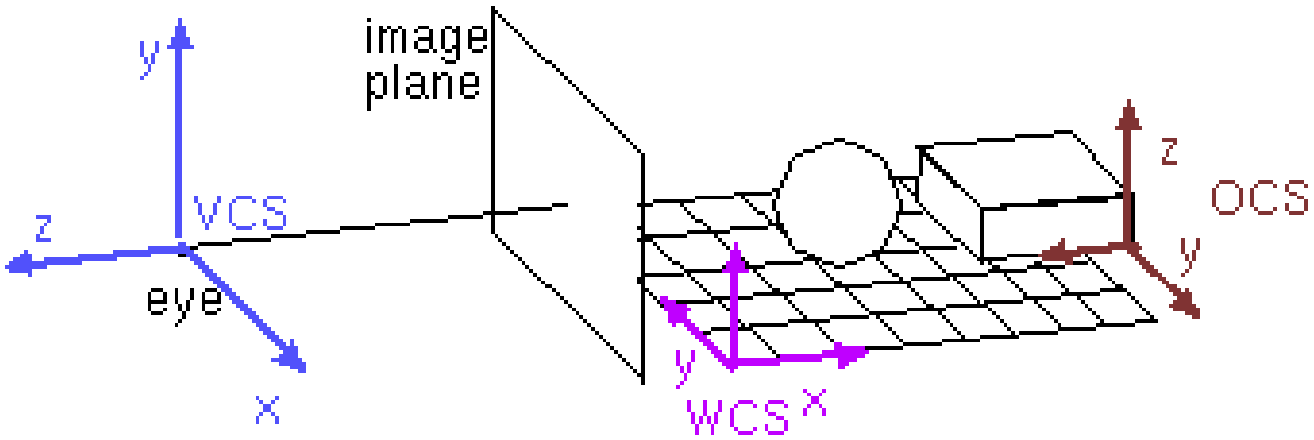


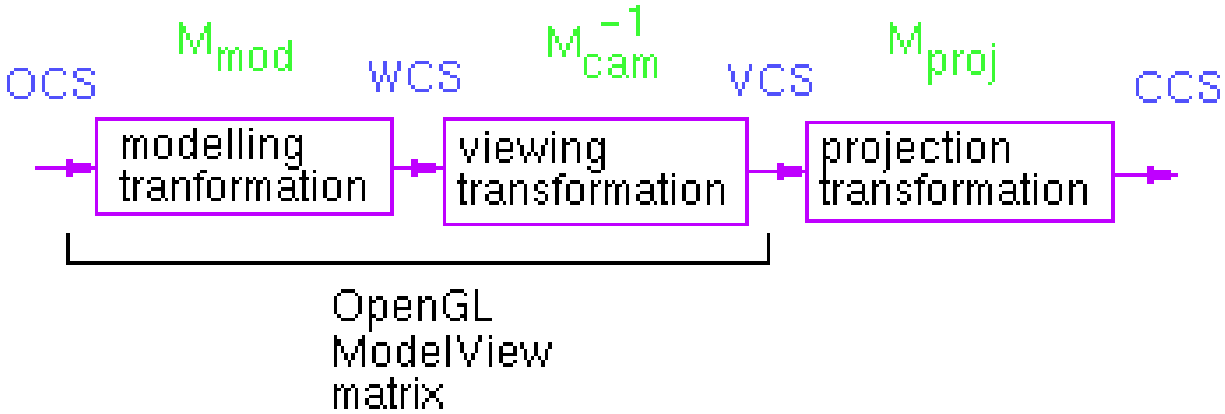
$$(\vec{a} \cdot \vec{w})\vec{a} = (\vec{a} \otimes \vec{a})\vec{w} = \begin{bmatrix} a_x^2 & a_x a_y & a_x a_z \\ a_y a_x & a_y^2 & a_y a_z \\ a_z a_x & a_z a_y & a_z^2 \end{bmatrix} \vec{w}$$

$$\vec{a} \times \vec{w} = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix} \vec{w} = C_a \vec{w}$$

$$T_{rot}(\vec{w}) = M_{rot}\vec{w} = (I\cos\theta + (\vec{a} \otimes \vec{a})(1 - \cos\theta) + C_a \sin\theta)\vec{w}$$

Преобразования в графическом конвейере OpenGL





Функции операций с матрицами преобразований в OpenGL

- Выбор матрицы преобразований для изменения:

```
void glMatrixMode(Glenum mode =
        {GL_MODELVIEW | GL_PROJECTION | GL_TEXTURE})
```

- Основные операции над матрицами:

```
void glLoadIdentity();
```

$$M = I$$

```
void glLoadMatrixd(GLdouble m[16]);

void glMultMatrixd(GLdouble m[16]);
```

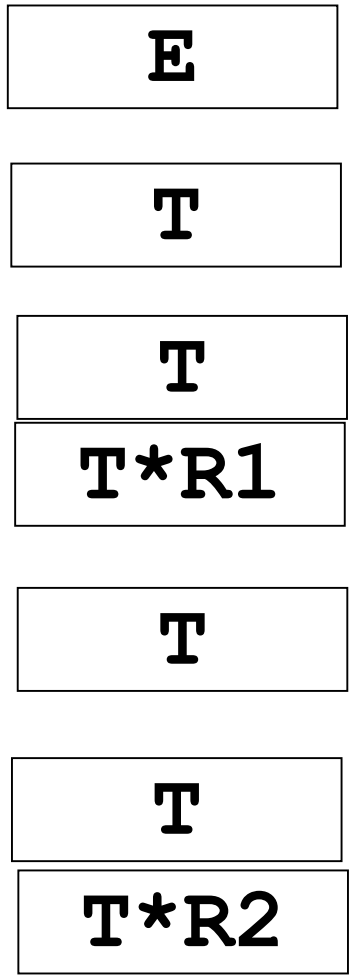
$$V = C \cdot \begin{bmatrix} m[0] & m[4] & m[8] & m[12] \\ m[1] & m[5] & m[9] & m[13] \\ m[2] & m[6] & m[10] & m[14] \\ m[3] & m[7] & m[11] & m[15] \end{bmatrix} \cdot \begin{bmatrix} v[0] \\ v[1] \\ v[2] \\ v[3] \end{bmatrix}$$

Стек матриц в OpenGL

```
glLoadIdentity();
glTranslated(…);

glPushMatrix();
glRotated(…);
glPopMatrix();

glPushMatrix();
glRotated(…);
glPopMatrix();
```



Определение глубины стека: glGetIntegerv (GL_MAX_MODELVIEW_STACK_DEPTH)

Определение текущей глубины стека: glGetIntegerv (GL_MODELVIEW_STACK_DEPTH)

Функции геометрических преобразований в OpenGL

```
void glTranslated(GLdouble x,
                  GLdouble y,
                  GLdouble z);

void glScaled(GLdouble x,
              GLdouble y,
              GLdouble z);

void glRotated(GLdouble angle,
               GLdouble ax,
               GLdouble ay,
               GLdouble az);
```

- преобразования объектов и камеры в OpenGL производятся с помощью умножения векторов координат на *текущую матрицу* в момент определения координат вершин
- при определении преобразования умножение матриц производится аналогично функции glMultMatrixd(), следовательно, преобразования необходимо задавать в обратном порядке.

Видовое преобразование в OpenGL

- стандартная камера в OpenGL:
  - наблюдатель в (0, 0, 0)
  - смотрит по направлению (0, 0, -1)
  - верх (0, 1, 0)

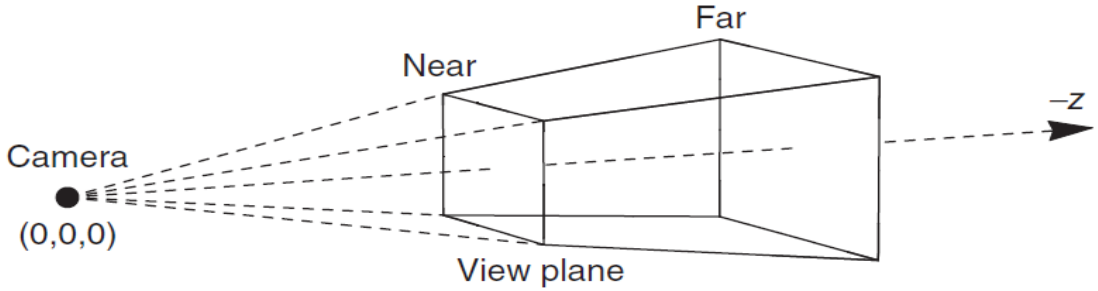
$$\text{gluLookAt}(\ eye_x, eye_y, eye_z, aim_x, aim_y, aim_z, up_x, up_y, up_z)$$

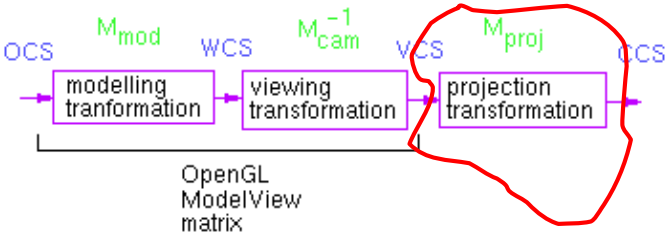
- eye – координаты наблюдателя
- aim – координаты «цели» (центра сцены)
- up – направление «вверх»

Вызывать команду gluLookAt() имеет смысл *перед* определением преобразований объектов, когда модельно-видовая матрица равна единичной

Проективное преобразование и проекция

- проективное преобразование: реализует 3D преобразование, подготавливая модель к переходу к 2D (координата Z сохраняется);
- проекция: переход к двумерной системе координат;
- после проективного преобразования необходимо отбросить координату z и получить значения в оконных координатах;
- объём видимости: ограничивает видимую область пространства;





Классификация проекций

- параллельная:
  - ортогональная, ортографическая*;
  - аксонометрическая:
    - прямоугольная * :
      - изометрия;
      - диметрия;
      - триметрия;
    - косоугольная **:
      - кавалье / военная перспектива (горизонтальная изометрия);
      - кабинетная (фронтальная диметрия);
- перспективная (центральная):
  - одноточечная;
  - двухточечная;
  - трехточечная;